

ZTF-Orchestrator

Field Installation Guide

Air-Gapped Deployment

Version 1.2.2 | 2026-05-21

Document Title	ZTF-Orchestrator Field Installation Guide
Version	1.2.2
Date	2026-05-21
Author	John Goulden
Classification	Public

Table of Contents

1. Overview	3
2. Prerequisites	3
3. Option A: Docker Deployment (Recommended).....	3
Step 1: Build machine (internet-connected)	3
Step 2: Transfer files	4
Step 3: Deployment host (air-gapped)	4
4. Option B: Standalone Python Deployment	4
Step 1: Build machine (internet-connected)	4
Step 2: Transfer files	5
Step 3a: Deployment host (Linux / macOS).....	5
Step 3b: Deployment host (Windows PowerShell).....	5
5. Option C: Internal Mirror Deployment.....	5
Step 1: Mirror repositories	5
Step 2: Mirror the Docker base image	6
Step 3: Point pip at your internal PyPI mirror	6
Step 4: Allow the internal ZTF mirror URL	6
6. First Login	6
Post-login checklist.....	6
7. Environment Variables	7
8. Internet Access Requirements	7
9. Port Reference.....	7
10. Troubleshooting	8
Log locations	8
Health check endpoint.....	8
Common issues.....	8

1. Overview

ZTF-Orchestrator is a web-based UI wrapper for the Nutanix ZeroTouch Framework (ZTF). It provides a visual operator interface for infrastructure automation workflows including cluster creation, imaging, Prism Central deployment, and workload provisioning.

Two components must be provisioned:

- ZTF-Orchestrator: the web UI (Flask backend, React frontend, this repository)
- ZeroTouch Framework: the Python automation engine called by the UI (github.com/nutanixdev/zerotouch-framework)

NOTE: Neither component requires internet access at runtime. All workflow API calls go directly to on-premises Nutanix infrastructure: Prism Central, Prism Element, and Foundation Central.

2. Prerequisites

Component	Requirement	Notes
Python	3.10 or later	Must be on system PATH
pip	Any version	Included with Python
Docker	20.10 or later	Option A (Docker) only
Docker Compose	v2 or later	Option A (Docker) only
git	Any version	Required on the build machine only not on the deployment host
Network	On-premises access to Nutanix PC/PE/FC	No internet access required at runtime

3. Option A: Docker Deployment (Recommended)

Build the Docker image on an internet-connected machine, transfer it as a compressed archive, then load and run on the air-gapped deployment host. This is the recommended approach for production and team deployments.

Step 1: Build machine (internet-connected)

Clone the repositories and build the image:

```
# Clone ZTF-Orchestrator
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git

# Build the Docker image
cd ZTF-Orchestrator
docker build -t ztf-orchestrator:1.2.2 .

# Clone the ZTF framework
git clone https://github.com/nutanixdev/zerotouch-framework.git \
/opt/zerotouch-framework
```

```
# Package for transfer
docker save ztf-orchestrator:1.2.2 | gzip > ztf-orchestrator-1.2.2.tar.gz
tar czf zerotouch-framework.tar.gz /opt/zerotouch-framework
```

Step 2: Transfer files

Transfer the following to the air-gapped deployment host via USB, secure copy, or approved media:

- ztf-orchestrator-1.2.2.tar.gz
- zerotouch-framework.tar.gz
- docker-compose.yml (from the ZTF-Orchestrator repository)

Step 3: Deployment host (air-gapped)

```
# Load the Docker image
docker load < ztf-orchestrator-1.2.2.tar.gz

# Extract the ZTF framework
tar xzf zerotouch-framework.tar.gz -C /opt/

# Start the service
ZTF_FRAMEWORK_PATH=/opt/zerotouch-framework docker compose up -d

# Retrieve first-run admin password from logs
docker compose logs | grep "Password:"
```

Open a browser and navigate to: <http://localhost:5001>

NOTE: The docker-compose.yml mounts /opt/zerotouch-framework as a read-only volume inside the container. No git, internet access, or build tools are required on the deployment host after this step.

4. Option B: Standalone Python Deployment

Use this option when Docker is not available on the deployment host. Python packages are downloaded as offline wheels on the build machine and installed without internet access on the deployment host.

Step 1: Build machine (internet-connected)

```
# Download Python wheels for ZTF-Orchestrator
pip download -r requirements.txt -d ./ztf-ui-wheels

# Clone ZTF framework and download its wheels
git clone https://github.com/nutanixdev/zerotouch-framework.git
pip download -r zerotouch-framework/requirements/prod.txt \
  -d ./ztf-framework-wheels

# Clone ZTF-Orchestrator
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git
```

Step 2: Transfer files

Transfer the following directories to the air-gapped deployment host:

- ZTF-Orchestrator/ (repository directory)
- zerotouch-framework/ (framework directory)
- ztf-ui-wheels/ (Python packages for the UI)
- ztf-framework-wheels/ (Python packages for ZTF)

Step 3a: Deployment host (Linux / macOS)

```
cd ZTF-Orchestrator
python3 -m venv venv
source venv/bin/activate
pip install --no-index --find-links=../ztf-ui-wheels -r requirements.txt

cd ../zerotouch-framework
pip install --no-index --find-links=../ztf-framework-wheels \
  -r requirements/prod.txt

cd ../ZTF-Orchestrator
ZTF_PATH=/path/to/zerotouch-framework python server.py
```

Step 3b: Deployment host (Windows PowerShell)

```
cd ZTF-Orchestrator
python -m venv venv
.\venv\Scripts\activate
pip install --no-index --find-links=../ztf-ui-wheels -r requirements.txt

cd ../zerotouch-framework
pip install --no-index --find-links=../ztf-framework-wheels `
  -r requirements\prod.txt

cd ../ZTF-Orchestrator
$env:ZTF_PATH = "C:\path\to\zerotouch-framework"
$env:ZTF_PORT = "8080" # if port 5001 is reserved by Windows/Hyper-V
python server.py
```

NOTE: ZTF ships a calm-whl/ directory of pre-bundled Calm DSL wheels. These cover Calm-specific dependencies without requiring PyPI access.

5. Option C: Internal Mirror Deployment

For environments with an internal Git server (Gitea / GitLab) and an internal package registry (Artifactory / devpi). This option enables fully automated deployments from internal infrastructure.

Step 1: Mirror repositories

- Mirror <https://github.com/VirtuArchitect/ZTF-Orchestrator> to your internal Git server
- Mirror <https://github.com/nutanixdev/zerotouch-framework> to your internal Git server

Step 2: Mirror the Docker base image

Mirror python:3.11-slim into your internal container registry, then update the Dockerfile:

```
# Dockerfile - line 1
FROM your-registry.internal/python:3.11-slim
```

Step 3: Point pip at your internal PyPI mirror

```
# Dockerfile - pip install step
RUN pip install --no-cache-dir \
    --index-url https://pypi.internal/simple \
    -r requirements.txt
```

Step 4: Allow the internal ZTF mirror URL

Add your internal mirror URL to the ALLOWED_REPOS set in server.py:

```
# server.py
ALLOWED_REPOS = {
    'https://github.com/nutanixdev/zerotouch-framework.git',
    'https://gitea.internal/nutanix/zerotouch-framework.git', # add this
}
```

Then update the ZTF Repository URL in the application Settings page to point at the internal mirror.

6. First Login

On first start, a default admin account is created automatically and credentials are printed to the terminal output or Docker logs.

```
=====
First run: default admin account created
Username: admin
Password: a3f8c2e1d94b7...
Change this immediately via Settings > Users.
=====
```

Post-login checklist

1. Copy the password from the terminal or docker compose logs output
2. Open <http://localhost:5001> (or the configured port) in a browser
3. Sign in with username admin and the printed password
4. Navigate to Users and create operator and viewer accounts as required
5. Navigate to Settings and verify the ZTF Installation Path points to the correct directory
6. Navigate to Setup & Install and run the Prerequisites Check to confirm the framework is detected

IMPORTANT: If the password was not captured: delete the users.json file from the data directory (~/.ztf-ui/users.json on Linux/macOS, C:\Users\<user>\.ztf-ui\users.json on Windows) and restart the server. A new admin account with a new password will be generated.

7. Environment Variables

All defaults work out of the box. Override via environment variables or a .env file. See .env.example in the repository for a complete template.

Variable	Default	Purpose
ZTF_DATA_DIR	~/.ztf-ui	Persistent data directory (users, history, configs, logs)
ZTF_PATH	~/zerotouch-framework	Path to the ZTF installation directory containing main.py
ZTF_PYTHON	python3	Python executable used to invoke ZTF scripts
ZTF_PORT	5001	Flask listen port binds to 127.0.0.1 only
ZTF_EXEC_TIMEOUT	3600	Maximum workflow execution time in seconds
ZTF_TOKEN_TTL	28800	Session token lifetime in seconds (default 8 hours)
ZTF_LOG_LEVEL	INFO	Log verbosity: DEBUG, INFO, WARNING, ERROR
ZTF_CONFIG_BACKUPS	5	Number of config file backup versions to retain per file
ZTF_WEBHOOK_URL	(empty)	Optional POST endpoint for execution completion notifications

8. Internet Access Requirements

Activity	Internet Required	Notes
Building the Docker image	Yes: one time only	On the build machine only
Downloading Python wheels	Yes: one time only	On the build machine only
Cloning the ZTF framework	Yes: one time only	On the build machine only
Running ZTF-Orchestrator	No	Fully offline at runtime
Executing workflows	No	ZTF calls Nutanix APIs on-premises only
Webhook notifications	Only if external URL configured	Optional feature can use internal endpoints

9. Port Reference

Port	Direction	Purpose
5001 (or ZTF_PORT)	Inbound to host	ZTF-Orchestrator web UI and API localhost only by default
9440	Outbound from host	Prism Central / Prism Element REST API
9440	Outbound from host	Foundation Central API
80 / 443	Outbound from host	Required on build machine only not on deployment host

NOTE: On Windows, port 5001 may fall within a Hyper-V reserved port range. If the server fails to start with a socket permissions error, set ZTF_PORT=8080 and open <http://localhost:8080> instead.

10. Troubleshooting

Log locations

- Standalone: ZTF_DATA_DIR/ztf-orchestrator.log (default: ~/.ztf-ui/ztf-orchestrator.log)
- Docker: `docker compose logs ztf-orchestrator`

Health check endpoint

The /health endpoint requires no authentication and can be used by monitoring tools:

```
GET http://localhost:5001/health

# Response
{"status": "ok", "version": "1.2.2", "ztf_installed": true}
```

Common issues

Symptom	Cause	Resolution
"Framework not found" badge	ZTF_PATH does not point to the ZTF directory	Verify ZTF_PATH is set to the directory containing main.py. Check Settings page.
"Port refused" on Windows	Port 5001 is in Hyper-V reserved range (4940-5039)	Set ZTF_PORT=8080 and access http://localhost:8080
"Permission denied" on log directory	Service user lacks write access to ZTF_DATA_DIR	Grant write permission: <code>chmod 700 ~/.ztf-ui</code> or run as correct user
First-run admin password not captured	Terminal output missed or Docker logs cleared	Delete users.json from ZTF_DATA_DIR and restart. New credentials will be printed.
"Connection refused" during dry run	Nutanix host IP unreachable from deployment host	Verify network routing from the deployment host to Prism Central / Foundation Central